

 OPTIMIZATION

10 Techniques That Make RAG **Faster** and More **Accurate**

Your RAG works.
Here's how to make it **actually good**.



AI Coach John

FOLLOW



PROFITBRIDGE
Drive Your Better Future

 Repost

Measure Before You Optimize

❗ THE CORE MISTAKE

Most engineers optimize the wrong thing. They fix latency when accuracy is the problem. They improve accuracy when users are churning due to speed.

The safe order: measure → identify biggest failure → fix that one thing → measure again.

OPTIMIZATION ROI OVERVIEW



★ **Rule:** Don't pick an optimization. Pick a measurement. The data tells you what to fix.



Al Coach John

FOLLOW



 **Repost**

Query Expansion - Multiple Perspectives

📌 WHAT IT IS

Generate 3–5 alternative phrasings of the user query, retrieve for each, merge results.

Embedding models work best when query language **matches document language**. Expansion closes that gap.

⚡ REAL SCENARIO

User asks: *"Can I cancel?"*

Expansions:

- "What is the cancellation policy?"
- "How do I terminate my subscription?"
- "What happens if I stop my plan?"

→ Together they retrieve **3x more** relevant chunks.

</> IMPLEMENTATION

```
from anthropic import Anthropic
client = Anthropic()

def expand_query(query, n=3):
    r = client.messages.create(
        model="claude-haiku-4-5-20251001",
        max_tokens=300,
        system=f"Generate {n} alt phrasings "
              "for search. Return JSON list.",
        messages=[{"role": "user",
                  "content": query}],
        )
    alts = json.loads(r.content[0].text)
    return [query] + alts

def retrieve_expanded(query):
    queries = expand_query(query)
    all_docs = []
    for q in queries:
        all_docs.extend(retriever.get(q))
    return deduplicate(all_docs)
```

 **Impact:** +10–20% accuracy on queries with terminology mismatches between user language and document language.



AI Coach John

FOLLOW



 **Repost**

Query Rewriting - Fix Before Retrieval

WHAT IT IS

Transform the raw conversational query into a standalone, retrieval-optimized query.

Critical for: Multi-turn conversations where "it" and "that" refer to previous context.

THE PROBLEM

Conversation:

User: "Tell me about RAG" → Bot explains

User: "What are its limitations?"

"its limitations" = meaningless to retriever.

Rewritten: "What are the limitations of Retrieval-Augmented Generation?" → perfect retrieval.

</> IMPLEMENTATION + HYDE

```
def rewrite_for_retrieval(
    query, chat_history
):
    history = "\n".join([
        f"{m['role']}:"
        for m in
        chat_history[-4:]
    ])
    return llm.generate(f"""
    Conversation: {history}
    Rewrite for semantic search,
    self-contained only.
    Query: {query}
    """)

# HyDE: embed a hypothetical
# answer
# instead of the query itself
def hyde_embed(query):
    answer = llm.generate(
        f"Write a short answer:
        {query}"
    )
    return embed(answer)

# Embeds the answer → finds doc
# language that matches answers
```

★ **Impact:** +10–20% accuracy. Low effort. Should be the first optimization every multi-turn RAG system adds.



AI Coach John

FOLLOW



Repost

Chunk Optimization - Most Impactful Parameter

Default 512-token chunks fail when answers span boundaries, chunks are too large, or document structure is lost.

</> 3 CHUNKING STRATEGIES

```
from langchain.text_splitter import (
    RecursiveCharacterTextSplitter,
    MarkdownHeaderTextSplitter,
    SemanticChunker
)
# Fixed-size (baseline)
fixed = RecursiveCharacterTextSplitter(
    chunk_size=512, chunk_overlap=50)

# Semantic (best accuracy)
semantic = SemanticChunker(
    embeddings=embeddings,

    breakpoint_threshold_type="percentile")

# Structure-aware (docs with headers)
md = MarkdownHeaderTextSplitter(
    headers_to_split_on=[
        ("##", "Section"),
        ("###", "Sub")])
```

📊 CHUNK SIZE VS ACCURACY

Size	Precision	Recall
128 tok	Very High	Low
256 tok	High	Medium
512 tok	Medium	High
1024 tok	Low	Very High
Semantic ✓	High	High

✅ RECOMMENDATION

Start with 512 tokens, 50 overlap.
Test semantic chunking. Pick by
eval score, not intuition. +15–25%
accuracy gain.



Hybrid Search - Semantic + Keyword

× SEMANTIC FAILS

User searches "Article 4(b)(ii)" - embedding has no idea what that means. Returns unrelated docs.

× KEYWORD FAILS

User searches "issues with my account" - no exact keyword match. Returns nothing relevant.

</> LANGCHAIN ENSEMBLE (RRF)

```
from langchain.retrievers import EnsembleRetriever, BM25Retriever

bm25 = BM25Retriever.from_documents(docs)
bm25.k = 10
vector = vectorstore.as_retriever(
    search_kwargs={"k": 10})

ensemble = EnsembleRetriever(
    retrievers=[bm25, vector],
    weights=[0.4, 0.6] # tune per use case
)
docs = ensemble.get_relevant_documents(query)
# Reciprocal Rank Fusion applied automatically
```

SEARCH ACCURACY COMPARISON



★ **Standard practice:** Every production RAG system should use hybrid search. +15–30% accuracy on mixed query types.



AI Coach John

FOLLOW



Repost

Re-ranking - Second Opinion on Relevance

⚡ STAGE 1 - BI-ENCODER (FAST)

Query + 10K docs



Top-20 results in 50ms

✓ STAGE 2 - CROSS-ENCODER (ACCURATE)

Query + Top-20 together



Top-3 (high precision)

</> CROSS-ENCODER RE-RANKING

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L-6-v2")

def two_stage_retrieve(query, final_k=3):
    # Stage 1: fast bi-encoder
    candidates = retriever.get(query, k=20)

    # Stage 2: cross-encoder re-rank
    pairs = [(query, doc.page_content)
              for doc in candidates]
    scores = reranker.predict(pairs)

    ranked = sorted(zip(candidates, scores),
                    key=lambda x: x[1], reverse=True)
    return [doc for doc, _ in ranked[:final_k]]
```

 Cohere Rerank API

 Voyage AI Rerank

 FlashRank (fast)

 ms-marco models



AI Coach John

FOLLOW



 **Repost**

Metadata Filtering - Fastest Optimization

📌 WHAT IT IS

Apply pre-filters based on structured metadata **before** vector search - smallest effort, biggest speed gain.

3 levels:

- > Hard filter: exact criteria match
- > Soft filter: boost matching docs
- > Dynamic: LLM extracts filters from query

⚡ SCENARIO

Legal firm with 50,000 docs across 10 clients. Without filter: Client A gets Client B's contracts. With filter: only Client A's 2024 contracts searched.

100% precision + 60% faster.

</> IMPLEMENTATION

```
# Hard filter (fastest)
results = vectorstore.search(
    query,
    filter={
        "department": "legal",
        "year": {"$gte": 2023}
    }
)

# Dynamic filter extraction
def extract_filters(query):
    return llm.generate(f"""
    Extract filters as JSON.
    Query: {query}
    Fields: dept, date, doc_type
    """)

filters = extract_filters(
    "Show legal contracts from
    2024")
# → {dept:"legal",
    year_from:2024}
```

PRECISION: WITHOUT VS WITH METADATA FILTER



AI Coach John

FOLLOW



🔄 Repost

Context Compression - Send Less, Get More

2,500

tokens without compression
(5 chunks × 500 tokens)



400

tokens after compression
84% reduction

</> LANGCHAIN COMPRESSION RETRIEVER

```
from langchain.retrievers.document_compressors import (
    LLMChainExtractor, EmbeddingsFilter)
from langchain.retrievers import (
    ContextualCompressionRetriever)

# Method 1: LLM extraction (high quality)
compressor = LLMChainExtractor.from_llm(llm)

# Method 2: Embeddings filter (fast, cheap)
emb_filter = EmbeddingsFilter(
    embeddings=embeddings,
    similarity_threshold=0.76)

retriever = ContextualCompressionRetriever(
    base_compressor=emb_filter,
    base_retriever=base_retriever)

# Only relevant sentences passed to LLM
docs = retriever.get_relevant_documents(query)
```

\$ Triple benefit: Lower token cost, faster inference, better answers (less noise for LLM to wade through).



AI Coach John

FOLLOW



Repost

Semantic Caching - Stop Paying Twice

30–50% of production queries are semantically duplicate. Cache those → cut latency from 1,200ms to <50ms.

IMPACT OF SEMANTIC CACHE HIT RATE



</> GPTCACHE IMPLEMENTATION

```
from gptcache import cache
from gptcache.manager import (
    CacheBase, VectorBase, get_data_manager)
from gptcache.similarity_evaluation import (
    SearchDistanceEvaluation)

cache.init(
    pre_embedding_func=get_prompt,
    embedding_func=onnx_embedding,
    data_manager=get_data_manager(
        CacheBase("sqlite"),
        VectorBase("faiss", dimension=512)),
    similarity_evaluation=SearchDistanceEvaluation(),
    similarity_threshold=0.85 # 85% sim = cache hit
)
# All LLM calls now auto-cached
result = llm.generate(query) # checks cache first
```



Parallel Retrieval - Stop Waiting in Line

⌚ SERIAL (SLOW)

Search 1: 100ms
 Search 2: 100ms
 Search 3: 100ms
 Search 4: 100ms
Total: 400ms

⚡ PARALLEL (FAST)

All 4 run together
 ← 100ms →

Total: 100ms (4x faster)

</> ASYNCIO.GATHER PATTERN

```
import asyncio

async def parallel_retrieve(query):
    results = await asyncio.gather(
        async_vector_search(query),          # Internal KB
        async_vector_search(query, "products"), # Product docs
        async_web_search(query),             # Live web
        async_bm25_search(query),            # Keyword
    )
    all_docs = [doc for r in results for doc in r]
    return deduplicate_by_content(all_docs)

# Serial:    4 × 100ms = 400ms ← old way
# Parallel:  max(100ms ...) = 100ms ← 4x faster
```

Impact: 40–60% latency reduction when querying multiple sources. Essential for hybrid knowledge systems.



AI Coach John

FOLLOW



Repost

Response Caching - The Fastest Response

The fastest response is a cached one. In FAQ-style systems, 60% of traffic is the same 50 questions.

</> TWO-TIER CACHING STRATEGY

```
import redis, hashlib

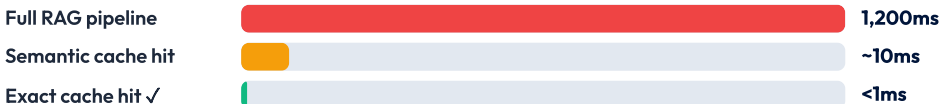
redis_client = redis.Redis()

def get_response(query):
    # Tier 1: Exact match (<1ms)
    q_hash = hashlib.md5(query.encode()).hexdigest()
    cached = redis_client.get(f"exact:{q_hash}")
    if cached:
        return cached.decode()

    # Tier 2: Semantic cache (~10ms)
    match = semantic_cache.lookup(query, threshold=0.95)
    if match:
        return match

    # Tier 3: Full RAG (~1,200ms)
    response = rag_pipeline(query)
    redis_client.setex(f"exact:{q_hash}", 3600, response)
    semantic_cache.store(query, response)
    return response
```

AVERAGE RESPONSE LATENCY



AI Coach John

FOLLOW



 **Repost**

In What Order to Optimize

📅 WEEK 1 - LOW EFFORT, HIGH IMPACT

- ✓ Metadata filtering - instant latency win
- ✓ Query rewriting - fixes conversational gaps
- ✓ Response caching for top-20 FAQ queries

📅 WEEK 2 - CORE RETRIEVAL QUALITY

- ✓ Chunk optimization - test 3 strategies
- ✓ Hybrid search - add BM25 alongside vector
- ✓ Re-ranking - cross-encoder, top-3 filter

📅 WEEK 3 - ADVANCED ACCURACY

- ✓ Query expansion - multi-query retrieval
- ✓ Context compression - token reduction

📅 WEEK 4 - SCALE & SPEED

- ✓ Semantic caching - 30-50% cost cut
- ✓ Parallel retrieval - multi-source fix

★ **The trio that fixes 60%:** Query rewriting + Metadata filtering + Re-ranking.
Start there.



Al Coach John

FOLLOW



↻ Repost

RAG Optimization Cheatsheet

TECHNIQUE	BENEFIT	EFFORT	IMPACT
Query Expansion	+Accuracy	Low	+10–20%
Query Rewriting	+Accuracy	Low	+10–20%
Chunk Optimization	+Accuracy	Medium	+15–25%
Hybrid Search	+Accuracy	Medium	+15–30%
Re-ranking	+Accuracy	Low	+10–25%
Metadata Filtering	+Speed	Low	–40–60% lat
Context Compression	+Accuracy +Cost	Medium	+5–15%
Semantic Caching	+Speed +Cost	Medium	–43% cost
Parallel Retrieval	+Speed	Medium	–40% lat
Response Caching	+Speed	Low	–80% lat

👉 Start with query rewriting + metadata filtering + re-ranking. That trio fixes 60% of production RAG problems.



AI Coach John

FOLLOW



↻ Repost